

Density-Based Classification From Unlabeled Data for Memory-Constrained Devices

Gianluca D’Altorio

Department of Computer Engineering Technology
Vanier College

I. PROPOSED PIPELINE

This section describes the part of the pipeline that runs on the implemented device itself. Everything else, grouping the training data into classes, building a density map for each class, and shrinking those maps down into small byte tables, happens beforehand on a regular computer and is assumed to already be completed. The device only ever sees the finished byte tables, never the original training data.

Algorithm 1 describes what the device does when it receives a new point to classify. First, the point is rescaled using the same bounds that were used during training. That scaled point is then used to figure out which cell of the grid it falls into. The device reads one stored number from that cell for each class, and picks whichever class has the highest number, unless even the highest number falls below a reject threshold τ to be trustworthy, in which case the device reports UNKNOWN instead of guessing. This last step matters: without it, the device would confidently name a class in places where it has never actually seen any data.

Since this whole process is just a few reads and comparisons, it costs the same amount of work no matter how much data was originally used to build the tables, which is why it’s cheap enough to run on a device with very little memory.

Algorithm 1 Table-lookup classification (online path).

Input: A raw query point (x, y)
Input: One byte table Q_c per class, each $G \times G$, values 0–255
Input: The scaling bounds $(x_{\min}, x_{\max})$, $(y_{\min}, y_{\max})$ used in training
Input: The grid size G and a reject threshold τ
Output: A predicted class $\hat{c}$, or UNKNOWN

```
1:  $\tilde{x} \leftarrow \frac{x - x_{\min}}{x_{\max} - x_{\min}}, \quad \tilde{y} \leftarrow \frac{y - y_{\min}}{y_{\max} - y_{\min}}$ 
2:  $\text{col} \leftarrow \lfloor \tilde{x} G \rfloor$ , clipped to  $[0, G - 1]$ 
3:  $\text{row} \leftarrow \lfloor \tilde{y} G \rfloor$ , clipped to  $[0, G - 1]$ 
4: for each class  $c$  do
5:    $v_c \leftarrow Q_c[\text{row}, \text{col}] / 255$  ▷ one byte read
6: end for
7:  $\hat{c} \leftarrow \arg \max_c v_c$  ▷ the winning class
8: if  $v_{\hat{c}} < \tau$  then
9:   return UNKNOWN ▷ outside every learned region
10: end if
11: return  $\hat{c}$ 
```

For this device, nearest-cell lookup (line 5 of Algorithm 1) is

preferred, rather than a bilinear blend of the four surrounding cells. Nearest-cell costs one byte read per class instead of four, and Section IV shows the two methods reach the same decision at a tested query point, so the extra cost of bilinear blending is not justified for this dataset.

II. APPLICATION

An example of this pipeline is implementation in a battery-powered wildlife trail camera that is deployed for extended, unattended periods of time in a forest. These devices run on small batteries for weeks or months, and every onboard component, including any classifier used to decide whether a trigger is worth recording, must fit within a few kilobytes of flash memory shared with the radio antennas, sensors, and sleep-cycle logic. A classifier costing 768 bytes rather than several kilobytes directly extends how long the device can run unattended.

Labeled training data is not available for this particular task. The motion, vibration, or infrared signal a sensor produces is specific to its installation site, the surrounding foliage, lighting, and typical wildlife at that location. Building a hand-labeled dataset would mean manually reviewing thousands of recorded trigger events, which is nearly impossible to repeat for every new deployment.

The classes that this device must distinguish are not naturally round. Realistic animal motion and branch motion produce different, irregular distributions in a feature space, such as motion duration versus motion intensity. Animal movement and wind noise both trace separate, differently shaped regions. A method such as k-means, which can only divide the feature space into straight-edged regions around a fixed number of centers, would misclassify motion near the boundary between these irregular shapes far more often than a density-based method that follows their actual contours would.

III. METHOD AND SETUP

The dataset used throughout this work is synthetic, generated with a fixed random seed. It consists of 1030 points in two dimensions: two crescent-shaped classes (340 and 300 points), one ring-shaped class (300 points), and 90 scattered background points intended to be discarded as noise.

The dataset was split into training (60%), validation (20%), and test (20%) sets using a fixed random seed. The training set builds the model: clustering, density fitting, and the discretized byte tables. The validation subset is used only to select the

density bandwidth. The test set is used exactly once, at the end, to report an honest measure of performance on data the model never saw.

Both features were scaled to the range $[0, 1]$ using the minimum and maximum values of the training subset alone. The same bounds are then applied to the validation and test sets, and would be shipped alongside the model to a real device.

Classes were discovered using DBSCAN with a search radius of $\varepsilon = 0.06$ and a minimum of 8 points per neighborhood, applied to the scaled training split (618 points, 60% of the full dataset). This recovered three clusters of sizes 209, 178, and 198 points, and labeled 33 points as noise, matching the three underlying shapes. A Gaussian kernel density estimate (KDE) was then fit to each discovered class using scikit-learn’s `KernelDensity`. The bandwidth was chosen by testing candidate values $h = \{0.03, 0.04, 0.05, 0.06, 0.08, 0.10, 0.12\}$ and selecting the value that maximized the mean log-likelihood of the held-out validation points. The algorithm selected $h = 0.05$.

Each class’s density map was evaluated on a fixed 16×16 grid, normalized so its own peak equals 1.0, and quantized to one byte (`uint8`) per cell. This gives a total shipped model size of 768 bytes for three classes, plus 16 bytes for the scaling bounds. At inference, a query point is classified by reading one byte per class from its corresponding grid cell and returning the class with the highest value, unless that value falls below a reject threshold of $\tau = 0.12$, in which case the result is UNKNOWN.

IV. EVIDENCE

The evidence for this pipeline follows the same structure as the data itself: built on the training split, tuned on the validation split, and confirmed on the test split once for the final measurement reported at the end of this section.

A. Classes are discoverable without labels

Figure 1 shows the training data clustered in two ways. DBSCAN recovers all three underlying shapes, two crescents and a ring, correctly labeling the scattered background points as noise. A k-means clustering given the correct number of classes cannot represent either shape correctly: it slices each crescent through its middle and places a center inside the ring’s empty hole, since it can only divide the feature space into straight-edged regions around a fixed number of centers. This result is the basis for choosing a density-based method over a center-based one.

B. The density maps are well-formed and survive storage

Figure 2 shows the continuous, per-class density maps produced by kernel density estimation on the training data, each normalized so its own peak equals 1.0. Figure 3 shows the same three maps combined into a single decision surface, with the reject threshold τ drawn as a plane passing through it: the two crescents form ridges above the plane, and the ring forms a crater whose center dips below it.

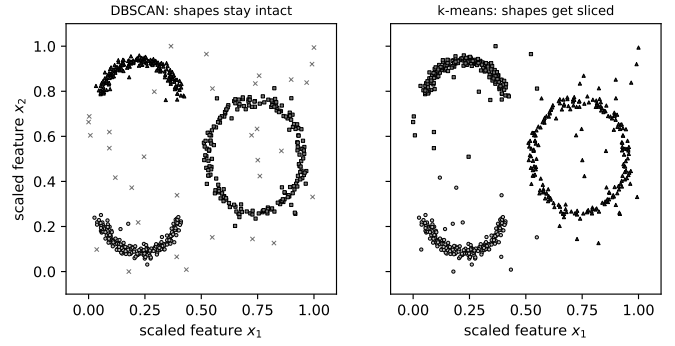


Fig. 1. DBSCAN discovers three classes from unlabeled training data, while k-means with the correct $k = 3$ slices each crescent and struggles to represent the ring.

Testing the bandwidth at its extremes shows the pipeline is sensitive to this choice. At $h = 0.03$, each class’s density map fragments into individual bumps around single training points rather than forming a smooth shape. At $h = 0.12$, the opposite failure occurs, as shown in Figure 4: the ring’s hollow center fills in with non-zero density as neighboring bumps merge, which would defeat the reject mechanism entirely for that region. The validation-selected value $h = 0.05$ avoids both failure modes.

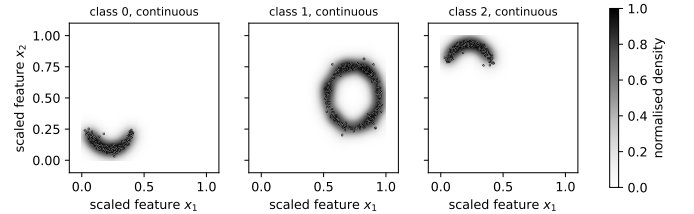


Fig. 2. Each discovered class’s density map, normalized to its own peak, before the lookup grid coarsens it.

Figure 5 shows the same three maps after being sampled onto the 16×16 grid and quantized to one byte per cell, the form that is actually shipped. The visible difference between Figures 2 and 5 is the coarseness of the grid, not the quantization itself. Rounding each value to one of 256 levels introduces a worst-case error of only 0.196% of each class’s peak, far too small to see.

C. What the model costs

Table I lists the flash cost of storing the maps at several precisions. Every row holds the same $G \times G$ values per class, only the bytes per cell differ. Storing the model as `uint8` costs 768 bytes for three classes, an eight times reduction versus `float64` at a measured worst-case accuracy cost of 0.196% of peak density. Section IV-F shows that this cost never changed a single classification decision on held-out data.

Doubling the grid size to $G = 32$ was also tested. This quadruples the footprint to 3072 bytes for three classes, while only doubling the grid’s linear resolution, since flash cost

Continuous decision surface ($\tau = 0.12$, $h = 0.05$)

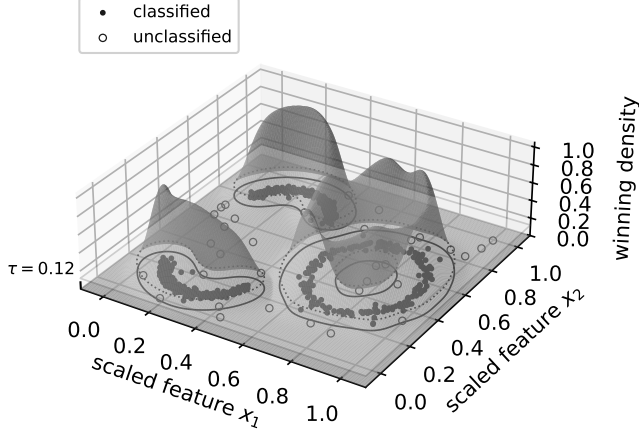


Fig. 3. The combined decision surface: at every location, the largest of the three normalized class densities, with the reject threshold $\tau = 0.12$ shown as a plane.

scales with G^2 while spatial detail scales with G . The ring's hollow center is visibly better resolved at this finer resolution, as shown in Figure 6, but the memory cost to achieve it grows much faster than the resolution improvement buys.

D. A single query, classified

Figure 7 shows one held-out test point classified by the stored model: the query falls into a single grid cell, and reading that cell's three stored bytes produces a confident decision for one class, with the other two classes reading zero. This is a representative example of a query landing well inside a learned region.

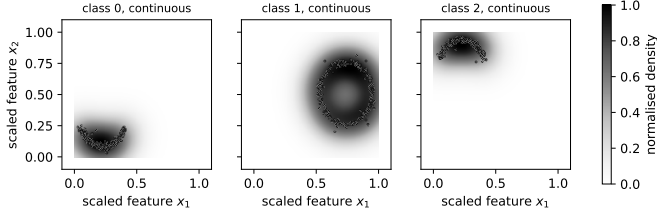


Fig. 4. Each discovered class's density map at $h = 0.12$, the largest bandwidth tested. Neighboring bumps merge and the ring's hollow center fills in with non-zero density, which would defeat the reject mechanism.

Comparing nearest-cell against bilinear lookup at this same query point shows the two methods agree. Nearest-cell reads three bytes directly from the query's cell, $(0, 255, 0)$, while bilinear blends the four surrounding cells to read $(0, 221.8, 0)$: a softer but still unambiguous decision for class 1. The stored bytes in the 3×3 neighborhood around this cell, for class 1, are:

$$\begin{pmatrix} 187 & 164 & 76 \\ 208 & 255 & 183 \\ 68 & 159 & 222 \end{pmatrix}$$

This illustrates how sharply the stored decision can change within a single grid cell, from the peak value of 255 down to 68 just one cell away. Since nearest-cell lookup costs K byte reads, versus bilinear's $4K$, and the two methods agree at this query, nearest-cell lookup is the more economical choice when the two do not disagree.

TABLE I
MODEL FOOTPRINT BY STORAGE FORMAT ($K = 3$ CLASSES, $G = 16$).

Storage	Bytes/cell	Per class	All K classes
uint8	1	256	768
uint16	2	512	1536
float32	4	1024	3072
float64	8	2048	6144

E. The reject threshold correctly reports unknown

Across the full 16×16 grid, 127 of 256 cells (49.6%) fall below the reject threshold $\tau = 0.12$, roughly half the feature space is correctly recognized as belonging to no learned class. Without this threshold, an $\arg \max$ classifier would report a confident-looking label at every one of these locations, including regions no training data ever touched, a failure that would not appear in an accuracy score computed only on points drawn from the three known shapes.

The clearest demonstration of this is the ring's own hollow center, at $(4.8, 1.6)$, a location entirely surrounded by ring training data on every side, yet containing none itself. Querying this point returns UNKNOWN, correctly distinguishing "near data" from "is data" at the exact query location, rather than assuming proximity to a class implies membership in it.

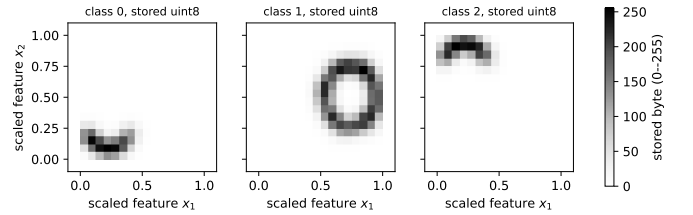


Fig. 5. The same three maps as Figure 2, stored as uint8 bytes, the form actually shipped to the device.

F. Agreement with the model on held-out data

The test split, held out from every prior stage, was used exactly once to compare the stored uint8 model's decisions against the full-precision continuous KDE's decisions on the same 206 points. The two models agreed on 100% of test points, including agreeing on which 6 points to reject as UNKNOWN (Table II). This confirms that the eightfold memory saving from quantization, measured in Section IV-C, cost nothing in terms of decisions on data the model never saw during training or tuning.

V. LIMITATIONS AND REMAINING UNCERTAINTY

The evidence in Section IV supports adopting this pipeline for the application described in Section II, but several limitations bound how far that evidence can be trusted.

A single synthetic dataset. Every result in this paper comes from one two-dimensional, synthetically generated dataset with a fixed random seed. The pipeline has not been tested on real sensor data, on data with more than two features, or on class shapes other than the crescents and ring used here. A deployment on real wildlife-sensor data would need to repeat this evaluation on the actual sensor’s feature space before any of these results could be assumed to transfer.

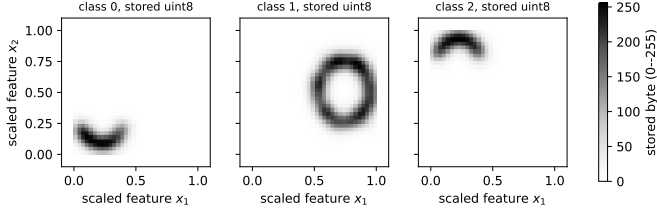


Fig. 6. The same `uint8` map stored as G^2 instead of G . The resolution improves, but memory cost increases.

The reject threshold was chosen, not derived. The value $\tau = 0.12$ was fixed as a constant in the pipeline rather than produced by any formal optimization or statistical procedure. It was observed to correctly reject the ring’s hollow center and roughly half of the full grid (49.6% of cells), but this behavior is specific to this dataset’s class shapes and this particular threshold value. A different application, or a different τ , could shift the reject region substantially, and there is no guarantee that 0.12 is the best choice for a different dataset.

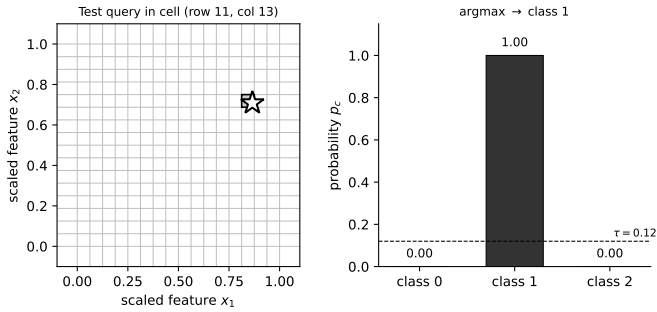


Fig. 7. One held-out test point classified: its grid cell (left) and the resulting per-class probabilities (right), with the winning class exceeding the reject threshold τ .

The test set result is an agreement, not accuracy. Since DBSCAN invented the class labels in Section IV-A, no independent ground truth exists for this dataset. The 100% figure reported in Section IV-F measures whether the cheap, stored model reproduces the decisions of the expensive, continuous model, not whether either model’s decisions are objectively correct. It is possible for both models to agree while both are wrong about the true underlying classes, if DBSCAN’s original clustering itself was flawed.

Grid resolution grows expensive quite quickly. Section IV showed that doubling the grid size from 16 to 32 quadrupled the flash footprint (from 768 to 3072 bytes) while only doubling the linear spatial resolution, since footprint scales with G^2 while detail scales with G . An application whose classes are finer-grained or more numerous than this one would hit its flash budget far sooner than this dataset suggests.

The result depends on a reasonable bandwidth. Testing the kernel density bandwidth at its extremes (Section IV-B) showed the pipeline degrades in both directions: too small a bandwidth fragments each class’s density map into individual bumps around single data points, while too large a bandwidth merges the ring’s hollow center into the surrounding density, defeating the reject mechanism entirely. The validation-selected value $h = 0.05$ avoided both failure modes here, but this value was tuned specifically to this dataset’s scale and point density, and is not guaranteed to transfer to a different deployment without repeating the same validation sweep.

TABLE II
CONFUSION BETWEEN THE CONTINUOUS KDE (ROWS) AND THE STORED `uint8` MODEL (COLUMNS) ON 206 HELD-OUT TEST POINTS.

	Class 0	Class 1	Class 2	Unknown
Class 0	69	0	0	0
Class 1	0	68	0	0
Class 2	0	0	63	0
Unknown	0	0	0	6

What to measure next. The most direct way to close these gaps would be to repeat this evaluation on real sensor data from the intended application, with a labeled subset to check the stored model’s decisions against true classes rather than only against the continuous model’s own output. A secondary measurement worth taking would be to sweep τ itself against that same labeled subset, to check whether 0.12 is actually a reasonable operating point or merely one that happened to work for this synthetic dataset.